Capítulo 5: Git en entornos distribuidos

Tutorial de Git basado en Pro Git (2.ª ed.) · Chacon y Straub · Elaborado por Ricardo · UNIAGRARIA

Introducción

Al ser distribuido, Git permite muchos modelos de colaboración. Este capítulo muestra los flujos de trabajo más comunes y cómo contribuir y mantener un proyecto.

5.1 Flujos de trabajo distribuidos

- Flujo centralizado: un único repositorio compartido al que todos hacen push; hay que hacer fetch/merge antes de publicar si alguien se adelantó.
- Flujo del gestor de integración: cada desarrollador publica en su repositorio propio (fork) y el mantenedor integra las contribuciones en el repositorio oficial. Es el modelo de GitHub.
- Flujo de dictador y tenientes: variante para proyectos enormes (como el kernel de Linux), con una jerarquía de responsables de subsistemas.

5.2 Contribuyendo a un Proyecto

Pautas para los commits

- Revisar antes de confirmar: git diff --check detecta errores de espacios en blanco.
- Hacer commits atómicos: cada uno con un cambio lógico y autocontenido.
- Mensaje: una línea de resumen de unos 50 caracteres, línea en blanco y luego un cuerpo que explique el porqué del cambio. Se recomienda el modo imperativo ("Agrega", "Corrige").

```
git diff --check
git add -p
git commit
```

Equipo privado pequeño

```
git fetch origin
git merge origin/main      # integrar antes de publicar
git push origin main
```

Proyecto público con fork

```
git clone <url-del-fork>
git switch -c arreglo-ortografia
# ... commits ...
git push -u origin arreglo-ortografia
# Se abre una solicitud de incorporación (pull request) desde la web
```

También es posible contribuir por correo electrónico con git format-patch y git send-email.

5.3 Manteniendo un proyecto

```
# Aplicar parches
git apply parche.diff
git am parche.mbox
```

```
# Traer ramas de otros desarrolladores
git remote add colaborador https://ejemplo.com/colaborador/proyecto.git
git fetch colaborador
git switch -c prueba colaborador/rama

# Revisar qué traen
git log main..prueba
git diff main...prueba
```

Publicar versiones

```
git tag -s v1.5 -m "Versión 1.5"        # etiqueta firmada
git describe main                        # nombre legible, p. ej. v1.5-3-g1a2b3c4
git archive main --prefix=proyecto/ | gzip > proyecto-1.5.tar.gz
git shortlog --no-merges main --not v1.4 # resumen de cambios para las notas
```

Resumen

Git admite flujos centralizados, de gestor de integración y de dictador con tenientes. Contribuir bien implica commits pequeños y bien descritos, integrar con los cambios remotos antes de publicar y, en proyectos públicos, trabajar con forks y ramas temáticas. Mantener un proyecto añade aplicar parches, revisar ramas ajenas y publicar versiones con etiquetas.

Referencia

Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress. https://git-scm.com/book/es/v2 — Capítulo 5.